第30章 多項式與FFT：詳解

30-E01 Degree Bound 與 Alias

推導

乘積degree≤5，故degree bound為6，至少6個相異點值才保證唯一。radix-2正2冪最小取8，兩個輸入都補到8，不能只補到各自的原長度。

1+x的2點DFT在1與−1是[2,0]，逐點平方為[4,0]。inverse為[(4+0)/2,(4−0)/2]=[2,2]。真正乘積1+2x+x^2在模x^2-1下有$x^2\equiv 1$，所以成2+2x；常數1和二次係數1被加到同一位置。

結論與檢核

degree≤2乘degree≤3需6點、radix-2取8。N=2的運算沒有算錯DFT，而是正確算了不同的循環卷積問題。

30-E02 同一 FFT 的全部中間值

推導

偶原索引0,2的係數[2,7]給[2+7,2−7]=[9,−5]；奇原索引1,3的[5,−1]給[5−1,5−(−1)]=[4,6]。bit-reversal後實體A=[2,7,5,−1]。

stage1,m=2：實體(0,1)用w=1，上9、下−5；實體(2,3)也w=1，上4、下6，A=[9,−5,4,6]。前者含原索引(0,2)，後者含原索引(1,3)，與實體索引不同。

stage2,m=4：實體(0,2)用w=1，u9,t4，寫A0=13,A2=5；實體(1,3)用w=i，u−5,t6i，寫A1=−5+6i,A3=−5−6i。按實體順序輸出[13,−5+6i,5,−5−6i]。

結論與檢核

y0=所有係數和13，y2=A(−1)=2−5+7+1=5，與結果一致。stage2配的是跨半塊(0,2)/(1,3)，不是再做兩個相鄰位置的蝶形。

30-E03 Inverse 的符號與正規化

推導

令F[k,j]=ω^{jk}、G[j,k]=ω^{-jk}/n。GF的(j,ℓ)是$\Sigma_k \omega^{k(l-j)}/n$，j=ℓ為1，其餘為0，所以G=$F^{-1}$。這同時確定反向指數與1/n，不是兩項互不相關的習慣。

F^2a/n在位置j收到$\Sigma_l a_l \cdot (\Sigma_k \omega^{k(j+l)}/n)$，只有$\ell\equiv -j \bmod n$留下。因此n4的a變[2,−1,7,5+i]，不是共軛[2,5−i,7,−1]。若正確採負號卻漏除4，得到[8,20+4i,28,−4]。

結論與檢核

錯符號造成循環索引反轉，漏正規化造成整體放大；用不對稱複數輸入可以把兩者與單純共軛區分清楚。

第30章 多項式與FFT：詳解 （續）

30-E04 插值的存在與唯一

推導

$\ell_k(x)=\Pi_{j\neq k}(x-x_j)/(x_k-x_j)$在$x_i$取$\delta_{ik}$，所以$\Sigma_k y_k\ell_k$匹配全部點且degree<n。若另有B也匹配，A−B的degree<n卻有n個相異根，由非零degree d至多d根的歸納定理，差必為零。

兩點0與1下，A_t(x)=1+2x+t・x(x−1)，t任意，都在0取1、1取3，degree<3。t不同則二次係數不同，故無限多解。若degree<2，x(x−1)項被排除，唯一為1+2x。

結論與檢核

插值唯一性要同時固定degree bound與足夠相異點。點值可能由許多高次多項式產生，不是只要有數據就能無條件唯一重建。

30-E05 卷積的三次轉換

推導

補零A=[1,2,0,0]，在1,i,−1,−i求值得[3,1+2i,−1,1−2i]；B=[1,−1,0,0]得[0,1−i,2,1+i]。逐點乘為[0,(1+2i)(1−i),−2,(1−2i)(1+i)]=[0,3+i,−2,3−i]。

負號DFT四項分別為4、4、−8、0；除4得[1,1,−2,0]，即$1+x-2x^2$。N4≥2+2−1，沒有高次繞回。

N8有$\log_2 8=3$stage，每stage8/2=4個butterfly，共12個。serial執行要做全部12個的算術；bounded-fan-in電路若同stage全並行、twiddle與接線已就緒，只有3stage的依賴深度。有限處理器或實體通訊不能直接由這張電路圖免費取得。

結論與檢核

三次轉換各Θ(NlogN)，點乘Θ(N)，總Θ(NlogN)。depth與work使用不同分母；少處理器時work/P下界仍存在。

30-E06 原地覆寫與 Stage 不變量

推導

正確上格9+4=13、下格9−4=5。若覆寫上格後才用它做減法，下格會錯成13−4=9。必須先u=A[top]、t=wA[bottom]，再寫A[top]=u+t與A[bottom]=u−t，兩者都讀同一舊u、t。

stage s的$m=2^s$；開始時每個m區塊左右半段各是m/2點的偶／奇DFT，合併後變完整m點DFT。s1的半段是單點DFT；歸納到m=n完成。固定m的實體間距m/2，twiddle腳標也須m，不能拿stage序號s替代。

第30章 多項式與FFT：詳解 （續）

n6不是2冪，半分會得到長3且下一層不再有整數兩等分；直接套這個程式不符合契約。可以使用其他radix／混合算法，或若目標是多項式普通卷積，改選足夠大的2冪補零；但把6點DFT輸入補到8，得到的是8點DFT，不能聲稱它就是原6點的同一組頻點。

結論與檢核

暫存變數是原地蝶形正確性的必要部分。補零能服務多項式乘法的點數要求，但一般訊號DFT長度改變就改變求值點，兩種任務不可混同。

30-E07 NTT 的條件與有號還原

推導

$13^2=169\equiv 16=-1 \bmod 17$，故$13^4\equiv 1$，且13、$13^2$都不為1，所以order恰4。
$4\cdot 13=52\equiv 1$，故$4^{-1}=13$；$\omega^{-1}=4$，因$13\cdot 4\equiv 1$。

NTT(A)=[3,10,16,9]，NTT(B)=[0,5,2,14]；乘積mod17為[0,50mod17,32mod17,126mod17]=[0,16,15,7]。用ω^{-1}與4^{-1}反轉得到[1,1,15,0]。因原乘積係數$\text{abs}\le \min(2,2)\cdot 2^2=8$，模17的每個剩餘類在[−8,8]內恰有一個整數代表，所以15對應−2。

若只保證$\text{abs}\le 20$，同一剩餘類15可對應−19、−2、15，三者都在界內，不能唯一還原。此時需要更大且有適合root的質數，或多模數配合足夠總模數與CRT。

結論與檢核

域中的NTT精確，不等於原整數值無繞回；還原前必須有幅度界。N可逆、root的order與有號幅度是三個各自必要的條件。

第30章 多項式與FFT：詳解 （續）

原創驗算程式附錄: complex_dft_roundtrip.py

```
"""Naive DFT and inverse DFT verify the FFT contract."""

import cmath

def dft(values, inverse=False):
    n = len(values)
    sign = -1 if inverse else 1
    result = []
    for k in range(n):
        total = 0j
        for j, value in enumerate(values):
            total += value * cmath.exp(sign * 2j * cmath.pi * j * k / n)
        result.append(total / n if inverse else total)
    return result


source = [1.0, 2.0, 0.0, -1.0]
expected = [2, 1+3j, 0, 1-3j]
assert max(abs(a-b) for a,b in zip(dft(source), expected)) < 1e-9
roundtrip = dft(dft(source), inverse=True)
assert max(abs(a - b) for a, b in zip(source, roundtrip)) < 1e-9
print([round(value.real, 9) for value in roundtrip])
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

第30章 多項式與FFT：詳解 （續）

原創驗算程式附錄: fft_depth_checks.py

```
"""Positive-exponent FFT, independently compared to direct evaluation/convolution."""
from cmath import exp
from math import pi
from itertools import product
import random


def bitreverse(index,bits):
    result=0
    for _ in range(bits):result=(result<<1)|(index&1);index>>=1
    return result


def iterative(values,inverse=False):
    n=len(values)
    if n==0 or n&(n-1):raise ValueError('radix2 length must be a positive power of2')
    bits=n.bit_length()-1;a=[complex(values[bitreverse(i,bits)]) for i in range(n)];stages=
[a[:]];wires=[]
    m=2
    while m<=n:
        wm=exp((-1 if inverse else 1)*2j*pi/m)
        for base in range(0,n,m):
            w=1
            for j in range(m//2):
                top,bottom=base+j,base+j+m//2;u=a[top];v=w*a[bottom]
                wires.append((m,top,bottom,w,u,v))
                a[top]=u+v;a[bottom]=u-v;w*=wm
        stages.append(a[:]);m*=2
    if inverse:a=[v/n for v in a]
    return a,stages,wires


def recursive(values,inverse=False):
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

第30章 多項式與FFT：詳解 （續）

```
    n=len(values)
    if n==0 or n&(n-1):raise ValueError('radix2 length must be a positive power of2')
    def transform(a):
        n=len(a)
        if n==1:return [complex(a[0])]
        e,o=transform(a[::2]),transform(a[1::2]);out=[0j]*n
        for k in range(n//2):
            term=exp((-1 if inverse else 1)*2j*pi*k/n)*o[k]
            out[k]=e[k]+term;out[k+n//2]=e[k]-term
        return out
    answer=transform(values)
    return [v/n for v in answer] if inverse else answer


def dft(values,inverse=False):
    n=len(values);sign=-1 if inverse else 1
    return [sum(a*exp(sign*2j*pi*j*k/n) for j,a in enumerate(values))/(n if inverse else 1)
for k in range(n)]


def close(a,b):return len(a)==len(b) and all(abs(x-y)<1e-8 for x,y in zip(a,b))


def convolution(a,b):
    if not a or not b:return []
    length=len(a)+len(b)-1;n=1
    while n<length:n*=2
    ya=iterative([*a,*([0]*(n-len(a)))])[0];yb=iterative([*b,*([0]*(n-len(b)))])[0]
    return iterative([u*v for u,v in zip(ya,yb)],True)[0][:length]


def direct_convolution(a,b):
    if not a or not b:return []
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

第30章 多項式與FFT：詳解 （續）

```
    answer=[0]*(len(a)+len(b)-1)
    for i,u in enumerate(a):
        for j,v in enumerate(b):answer[i+j]+=u*v
    return answer


def ntt(values,prime,root,inverse=False):
    n=len(values)
    assert pow(root,n,prime)==1
    assert all(pow(root,k,prime)!=1 for k in range(1,n))
    omega=pow(root,-1,prime) if inverse else root
    answer=[sum(v*pow(omega,j*k,prime) for j,v in enumerate(values))%prime for k in range(n)]
    return [(v*pow(n,-1,prime))%prime for v in answer] if inverse else answer


def main():
    transforms=0;products=0
    for n,alphabet in [(1,range(-2,3)),(2,range(-2,3)),(4,range(-2,3)),(8,[-1,0,1])]:
        for values in product(alphabet,repeat=n):
            expected=dft(values);actual,stages,wires=iterative(values)
            assert close(actual,expected) and close(recursive(values),expected)
            assert close(iterative(actual,True)[0],values) and close(dft(actual,True),values)
            assert len(wires)==n//2*(n.bit_length()-1)
            transforms+=1
    rng=random.Random(30)
    for n in [16,32,64]:
        for _ in range(30):
            values=[complex(rng.randrange(-9,10),rng.randrange(-9,10)) for _ in range(n)]
            assert close(iterative(values)[0],dft(values));transforms+=1
    polys=[values for n in range(5) for values in product([-1,0,1],repeat=n)]
    for a in polys:
        for b in polys:
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

第30章 多項式與FFT：詳解 （續）

```
            assert close(convolution(a,b),direct_convolution(a,b));products+=1
    result,stages,wires=iterative([2,5,7,-1])
    assert close(stages[0],[2,7,5,-1]) and close(stages[1],[9,-5,4,6])
    assert [(u,v) for m,u,v,*_ in wires if m==4]==[(0,2),(1,3)]
    assert close(result,[13,-5+6j,5,-5-6j])
    assert close(iterative([1,2,0,-1])[0],[2,1+3j,0,1-3j])
    ya=dft([1,2,0,0]);yb=dft([1,-1,0,0]);combined=[a*b for a,b in zip(ya,yb)]
    assert close(ya,[3,1+2j,-1,1-2j]) and close(yb,[0,1-1j,2,1+1j])
    assert close(combined,[0,3+1j,-2,3-1j]) and close(dft(combined,True),[1,1,-2,0])
    # Underpadding produces cyclic convolution, not the ordinary product.
    shorta=dft([1,1]);short=[v*v for v in shorta]
    assert close(dft(short,True),[2,2])
    na=ntt([1,2,0,0],17,13);nb=ntt([1,-1,0,0],17,13)
    assert na==[3,10,16,9] and nb==[0,5,2,14]
    assert [(a*b)%17 for a,b in zip(na,nb)]==[0,16,15,7]
    assert ntt([0,16,15,7],17,13,True)==[1,1,15,0]
    for bad in [[],[1,2,3]]:
        try:iterative(bad)
        except ValueError:pass
        else:raise AssertionError('invalid radix2 input accepted')
    # Wrong positive sign in an inverse gives circular index reversal, not a general
conjugation.
    asymmetric=[2,5+1j,7,-1]
    assert close([v/4 for v in iterative(iterative(asymmetric)[0])[0]],
                 [asymmetric[0],asymmetric[3],asymmetric[2],asymmetric[1]])
    print(f'PASS FFT_direct_evaluation_cases={transforms} polynomial_convolution_cases=
{products} '
          'recursive_and_iterative_and_inverse=True distinct_stage_values_and_wires=True '
          'zero_padding_alias=2,2 NTT_mod17_exact=1,1,15,0
inverse_wrong_sign=index_reversal')


if __name__=='__main__':main()
```

第30章 多項式與FFT：詳解 （續）

原創驗算程式附錄: fft_trace.py

```
"""Verify the required radix-2 FFT stages and inverse round trip."""

import cmath


def bit_reverse_two_bits(value):
    return ((value & 1) << 1) | ((value >> 1) & 1)


def staged_fft(values, inverse=False):
    n = len(values)
    assert n == 4
    stages = []
    a = [values[bit_reverse_two_bits(i)] for i in range(n)]
    origins = [bit_reverse_two_bits(i) for i in range(n)]
    for half in (1, 2):
        block = 2 * half
        root = cmath.exp((2j if not inverse else -2j) * cmath.pi / block)
        for start in range(0, n, block):
            weight = 1
            for j in range(half):
                u = a[start + j]
                t = weight * a[start + j + half]
                a[start + j] = u + t
                a[start + j + half] = u - t
                weight *= root
        stages.append(tuple(a))
    if inverse:
        a = [item / n for item in a]
    return a, stages, origins


def close_pair(actual, expected):
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

第30章 多項式與FFT：詳解 （續）

```
    return all(abs(x - y) < 1e-9 for x, y in zip(actual, expected))


def direct_dft(values, inverse=False):
    n = len(values)
    sign = -1 if inverse else 1
    return [
        sum(values[j] * cmath.exp(sign * 2j * cmath.pi * j * k / n) for j in range(n)) / (n
if inverse else 1)
        for k in range(n)
    ]


def run():
    source = [1, 2, 0, -1]
    output, stages, origins = staged_fft(source)
    assert origins == [0, 2, 1, 3]
    assert close_pair(stages[0], [1, 1, 1, 3])
    expected = [2, 1 + 3j, 0, 1 - 3j]
    assert close_pair(output, expected)
    assert close_pair(output, direct_dft(source))
    assert close_pair(direct_dft(direct_dft(source), inverse=True), source)
    recovered, inverse_stages, _ = staged_fft(output, inverse=True)
    assert close_pair(recovered, source)
    assert len(stages) == 2 and len(inverse_stages) == 2
    print("PASS fft origins=0,2,1,3 stage1=1,1,1,3 output=2,1+3i,0,1-3i roundtrip=yes")
    return output


if __name__ == "__main__":
    run()
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

參考資料與署名

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study